联邦学习医学图像生成系统 (Federated Medical Image Generation System)

目录

- 项目概述
- 整体架构
- 目录结构
- 核心模块详解
  - 配置模块
  - 服务器模块
  - 客户端模块
  - 适配器模块
  - 控制算法模块
  - 统计模块
  - 工具模块
  - 推理生成模块
- 超参数详解
- 数据流与通信协议
- 使用指南
- 依赖环境

项目概述

本项目是一个基于联邦学习（Federated Learning）的医学图像生成系统，采用扩散模型（Diffusion Model）作为核心生成架构。系统实现了自适应本地迭代次数的联邦优化算法，专门针对医学CT/MRI图像的生成任务进行优化。

核心特性

1. **联邦学习架构**：支持多客户端分布式训练，保护数据隐私
2. **自适应迭代控制**：基于收敛界优化的本地迭代次数自适应调整
3. **医学图像适配器**：针对医学图像特征设计的轻量级适配器模块
4. **动量加速优化**：采用Heavy Ball动量梯度下降加速收敛
5. **能耗与时间优化**：考虑通信能耗和计算时间的资源优化

整体架构

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                联邦学习系统架构                                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                          服务器端（server.py）                          │   │
```

```
│   │   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────────────┐   │   │
│   │   │   全局适配器     │   │   控制算法       │   │     统计收集模块         │   │   │
│   │   │   (Adapter)      │   │  (AdaptiveTau)   │   │   (CollectStatistics)    │   │   │
│   │   └──────────────────┘   └──────────────────┘   └──────────────────────────┘   │   │
│   │                                  │                                              │   │
│   │                  ┌───────────────┼───────────────┐                              │   │
│   │                  ▼               ▼               ▼                              │   │
│   │            参数聚合          tau计算          损失追踪                           │   │
│   └──────────────────────────────────────────────────────────────────────────────────┘   │
│                                  │ Socket通信                                            │
│                  ┌───────────────┼───────────────┐                                       │
│                  ▼               ▼               ▼                                       │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐                   │
│   │   客户端 1       │   │   客户端 2       │   │   客户端 N       │                   │
│   │  (client.py)     │   │  (client.py)     │   │  (client.py)     │                   │
│   │                  │   │                  │   │                  │                   │
│   │  ┌────────────┐  │   │  ┌────────────┐  │   │  ┌────────────┐  │                   │
│   │  │本地适配器  │  │   │  │本地适配器  │  │   │  │本地适配器  │  │                   │
│   │  └────────────┘  │   │  └────────────┘  │   │  └────────────┘  │                   │
│   │  ┌────────────┐  │   │  ┌────────────┐  │   │  ┌────────────┐  │                   │
│   │  │ UNet模型   │  │   │  │ UNet模型   │  │   │  │ UNet模型   │  │                   │
│   │  └────────────┘  │   │  └────────────┘  │   │  └────────────┘  │                   │
│   │  ┌────────────┐  │   │  ┌────────────┐  │   │  ┌────────────┐  │                   │
│   │  │本地数据集  │  │   │  │本地数据集  │  │   │  │本地数据集  │  │                   │
│   │  └────────────┘  │   │  └────────────┘  │   │  └────────────┘  │                   │
│   └──────────────────┘   └──────────────────┘   └──────────────────┘                   │
│                                                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────┘
```

训练流程

```
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│                                          训练流程图                                      │
├──────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                          │
│   1. 初始化阶段                                                                          │
│       ├── 服务器初始化全局适配器参数                                                      │
│       ├── 客户端连接服务器                                                                │
│       └── 服务器分发初始参数给所有客户端                                                  │
│                                                                                          │
│   2. 训练循环（每轮）                                                                    │
│       ├── 服务器发送全局参数和tau配置                                                     │
│       ├── 客户端本地训练                                                                  │
│       │   ├── 加载医学图像数据                                                            │
│       │   ├── 生成带噪潜变量                                                              │
│       │   ├── 通过适配器处理UNet特征                                                      │
│       │   ├── 计算噪声预测损失                                                            │
│       │   └── 反向传播更新本地适配器                                                      │
│       ├── 客户端发送更新后的参数和梯度                                                    │
│       ├── 服务器聚合参数                                                                  │
│       │   ├── 按数据量加权平均                                                            │
```

```
│      │    ├── 检查NaN值                                                    │
│      │    └── 更新最小损失模型                                              │
│      └── 控制算法计算下一轮tau                                              │
│                                                                            │
│  3. 终止条件                                                               │
│      ├── 达到最大训练轮数                                                   │
│      ├── 达到时间/能量上限                                                  │
│      └── 收敛（连续N轮损失未改善）                                          │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

目录结构

```
jishe/
├── config.py                    # 全局配置文件（参数定义、工具函数）
├── config.yaml                  # YAML配置文件（可配置的超参数）
├── server.py                    # 服务器端主程序
├── client.py                    # 客户端主程序
├── Adapter.py                   # 适配器模块定义
├── start_training.py            # 训练启动脚本
├── prompt.txt                   # 训练文本提示
├── requirements.txt             # 依赖包列表
│
├── control_algorithm/           # 控制算法模块
│   ├── __init__.py
│   └── adaptive_tau.py          # 自适应tau控制算法
│
├── statistic/                   # 统计模块
│   ├── __init__.py
│   └── collect_stat.py          # 统计数据收集
│
├── util/                        # 工具模块
│   ├── __init__.py
│   ├── utils.py                 # 通用工具函数
│   ├── sampling.py              # 小批量采样
│   └── time_generation.py       # 时间生成
│
├── apply/                       # 推理生成模块
│   └── gen.py                   # 图像生成脚本
│
└── scripts/                     # 脚本目录
    ├── train.sh                 # 训练脚本
    └── generate.sh              # 生成脚本
```

核心模块详解

1. 配置模块

文件：`config.py`

功能概述：全局配置管理，包含所有超参数定义、模型路径配置、工具函数实现。

核心组件：

1.1 配置加载函数

```
def load_config(config_path="config.yaml")
```

- **功能**：从YAML文件加载配置
- **参数**：`config_path` - 配置文件路径
- **返回**：配置字典

1.2 服务器配置参数

参数名	类型	默认值	说明
`SERVER_ADDR`	str	"localhost"	服务器地址
`SERVER_PORT`	int	5100	服务器端口
`n_nodes`	int	4	客户端数量
`max_training_rounds`	int	200	最大训练轮数
`momentum_value`	float	0.9	动量系数
`dataset_root`	str	"data/medical_images"	数据集根目录

1.3 训练配置参数

参数名	类型	默认值	说明
`moving_average_holding_param`	float	0.9	移动平均保持系数
`single_run`	bool	True	是否单次运行
`use_min_loss`	bool	True	是否使用最小损失策略
`read_all_data_for_stochastic`	bool	True	是否预读所有数据
`MAX_CASE`	int	4	最大case数量
`tau_max`	int	100	tau最大值
`control_param_phi`	float	0.00005	控制参数φ

1.4 物理仿真参数

参数名	类型	默认值	说明

参数名	类型	默认值	说明
`N0_dBm`	int	-174	噪声功率谱密度 (dBm/Hz)
`N0`	float	计算值	噪声功率谱密度 (W/Hz)

1.5 随机参数生成函数

```
def generate_random_params(n_nodes, round_num)
```

功能：为每轮训练生成随机通信和计算参数

生成的参数：

参数	范围	单位	说明
`b` (带宽)	0.5~1 MHz	Hz	子信道带宽，总和限制3MHz
`p_dBm` (发射功率)	5~20 dBm	dBm	发射功率
`p` (发射功率)	计算值	W	发射功率（瓦特）
`f` (CPU频率)	0.5~1 GHz	Hz	CPU频率
`kappa` (能耗系数)	1e-28~5e-28	$J \cdot s^2/cycle^2$	计算能耗系数
`o` (CPU周期)	3e7~5e7	cycles	单样本处理CPU周期
`PL` (路径损耗)	计算值	dB	路径损耗
`g` (信道增益)	计算值	-	信道增益（线性值）
`r` (传输速率)	计算值	bps	香农容量

信道模型：

- 用户位置：500m × 500m 区域内随机分布
- 基站位置：中心 (250, 250)
- 路径损耗公式：`PL = 128.1 + 37.6 * log10(distance_km)`

1.6 时间能耗计算函数

```
def calculate_time_energy(tau, C, sample_size, f, kappa)
```

功能：计算客户端迭代时间和能耗

公式：

- 迭代时间：`t = tau * C * sample_size / f`
- 能耗：`E = kappa * tau * sample_size * C * f²`

1.7 UNet层自动检测函数

```
def get_adapt_layers(unet=None)
```

- **功能**：自动检测UNet上采样层数，返回最接近输出的两层
- **返回**：层名列表，如 `["up2", "up3"]`

```
def get_img_dims_per_level(unet=None)
```

- **功能**：自动检测UNet上采样层的输出维度
- **返回**：维度字典，如 `{"up0": 1280, "up1": 1280, "up2": 640, "up3": 320}`

1.8 命令行参数解析

```
def parse_args()
```

支持的命令行参数：

参数名	类型	默认值	说明
`--base_model_path`	str	"model"	基础模型路径
`--revision`	str	None	模型版本
`--unet_subfolder`	str	"unets/4/unet"	UNet子目录
`--text_encoder_subfolder`	str	"text_encoder"	文本编码器子目录
`--vae_subfolder`	str	"vae"	VAE子目录
`--scheduler_subfolder`	str	"scheduler"	调度器子目录
`--tokenizer_subfolder`	str	"tokenizer"	分词器子目录
`--output_dir`	str	"output/"	输出目录
`--resolution`	int	512	图像分辨率
`--train_batch_size`	int	16	训练批次大小
`--num_train_epochs`	int	100	训练轮数
`--gradient_accumulation_steps`	int	1	梯度累积步数
`--gradient_checkpointing`	bool	False	是否使用梯度检查点
`--learning_rate`	float	0.0001	学习率
`--mixed_precision`	str	None	混合精度模式 (no/fp16/bf16)

参数名	类型	默认值	说明
`--checkpointing_steps`	int	100	检查点保存步数

文件：`config.yaml`

功能概述：YAML格式的可配置参数文件

配置结构：

```yaml
model:
  base_model_path: "model"            # 基础模型路径
  unet_subfolder: "unets/4/unet"      # UNet子目录
  text_encoder_subfolder: "text_encoder"
  vae_subfolder: "vae"
  scheduler_subfolder: "scheduler"
  tokenizer_subfolder: "tokenizer"
  output_dir: "output/"               # 输出目录
  resolution: 512                     # 图像分辨率
  train_batch_size: 8                 # 训练批次大小
  num_train_epochs: 100               # 训练轮数
  gradient_accumulation_steps: 1      # 梯度累积步数
  gradient_checkpointing: false       # 梯度检查点
  learning_rate: 0.0003               # 学习率
  mixed_precision: null               # 混合精度
  checkpointing_steps: 100            # 检查点保存步数

server:
  server_addr: "localhost"            # 服务器地址
  server_port: 5100                   # 服务器端口
  n_nodes: 4                          # 客户端数量
  momentum_value: 0.9                 # 动量值
  dataset_root: "dataset_case4"       # 数据集根目录
  max_training_rounds: 200            # 最大训练轮数
  timestep_ranges:                    # 时间步范围配置
    - range: [500, 601]               # 时间步范围
      adapter:
        hidden_layers: 2              # 隐藏层层数
        hidden_size: [256, 128]       # 隐藏层大小
        activation: "gelu"            # 激活函数

client:
  dataset_path: "dataset_case4"       # 客户端数据集路径

adapter:
  hidden_layers: 2                    # 隐藏层层数
  hidden_size: [256, 128]             # 隐藏层大小
  activation: "gelu"                  # 激活函数

training:
  use_adapt_local: true               # 是否使用自适应本地迭代
```

```
    moving_average_holding_param: 0.9  # 移动平均保持参数
    stop_conditions:
      time_limit: null                 # 时间上限（秒）
      energy_limit: null               # 能量上限（焦耳）
      convergence_rounds: 20           # 收敛轮数
      loss_improvement_percentage: 1.0 # 损失改善百分比阈值

  text:
    training_text: "COVID-19 chest CT: ..."  # 训练文本
```

---

2. 适配器模块

文件：`Adapter.py`

功能概述：定义图像适配器，用于在UNet上采样层注入医学图像结构特征。

类：`ImageAdapter`

```
class ImageAdapter(nn.Module)
```

功能：处理UNet上采样层特征，注入医学图像结构

初始化参数：

参数	类型	说明
`img_dim`	int	输入图像特征维度

内部结构：

```
ImageAdapter
├── range_adapters: List[Dict]     # 每个时间步范围的适配器
│   └── adapter_dict
│       ├── range: [start, end]    # 时间步范围
│       ├── structural_encoder     # 结构编码器 (Sequential)
│       │   ├── Conv2d(1x1)        # 通道调整
│       │   ├── Activation         # 激活函数
│       │   ├── Conv2d(3x3)        # 特征提取
│       │   ├── Activation
│       │   └── ... (隐藏层)
│       └── feature_proj           # 特征投影
│           └── Conv2d(1x1)        # 输出投影
└── kappa_scale: int = 1           # kappa缩放系数
```

适配器网络配置：

配置项	可选值	说明
`hidden_layers`	1, 2, 3, ...	隐藏层层数
`hidden_size`	int 或 List[int]	隐藏层大小
`activation`	"relu", "gelu", "sigmoid", "tanh"	激活函数

前向传播:

```
def forward(self, img_feat, timestep=None, kappa=None)
```

参数:

- `img_feat`: 输入图像特征 [B, C, H, W]
- `timestep`: 当前时间步
- `kappa`: 适配强度系数

输出:

- 适配后的特征: `(1 - kappa) * img_feat + kappa * adapted_feat`

kappa处理逻辑:

1. 检查时间步是否在配置范围内
2. 如果不在范围内，直接返回原始特征
3. 如果kappa为None，使用默认值0.5
4. 将kappa限制在 [0.1, 0.8] 范围内

---

3. 服务器模块

文件: `server.py`

功能概述: 联邦学习服务器端主程序，负责参数聚合、控制算法执行、统计收集。

主要组件:

3.1 初始化阶段

```
# 服务器Socket初始化
listening_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
listening_sock.bind((SERVER_ADDR, SERVER_PORT))

# 等待客户端连接
while len(client_sock_all) < n_nodes:
    listening_sock.listen(5)
    (client_sock, (ip, port)) = listening_sock.accept()
    client_sock_all.append(client_sock)
```

3.2 全局适配器初始化

```
adapter_global = nn.ModuleDict()
for level in adapt_layers:
    adapter_global[level] = nn.ModuleDict({
        "img_adapter": ImageAdapter(img_dim=img_dims_per_level[level])
    })
adapter_global["img_kappa_hypernet"] = ImageKappaHyperNetwork(
    text_dim=768,
    time_dim=512,
    level_embed_dim=32,
    levels=adapt_layers
)
```

3.3 类：`ImageKappaHyperNetwork`（服务器端版本）

```
class ImageKappaHyperNetwork(nn.Module)
```

功能：图像专用kappa超网络，预测图像适配强度

架构：

```
ImageKappaHyperNetwork
├── text_dim: int = 768            # 文本嵌入维度
├── time_dim: int = 512            # 时间嵌入维度
├── level_embed_dim: int = 32      # 层级嵌入维度
├── levels: List[str]              # 层级列表
│
├── global_encoder: Sequential     # 全局编码器
│   ├── Linear(768+512 → 512)
│   ├── GELU()
│   └── Dropout(0.1)
│
├── level_predictors: ModuleDict   # 层级预测器
│   └── [level]: Sequential
│       ├── Linear(512+32 → 128)
│       ├── GELU()
│       ├── Dropout(0.1)
│       ├── Linear(128 → 1)
│       └── Sigmoid()
│
└── level_embeddings: ParameterDict # 层级嵌入
    └── [level]: Parameter(32)
```

前向传播：

```
def forward(self, text_emb, timestep, current_level=None)
```

输入:

- `text_emb`: 文本嵌入 [B, seq_len, 768]
- `timestep`: 时间步 [B]
- `current_level`: 当前层级名（可选）

输出:

- 如果指定层级：返回该层级的kappa值 [B]
- 否则：返回所有层级的kappa字典

3.4 时间步嵌入函数

```
def get_timestep_embedding(timesteps, embedding_dim)
```

功能：将时间步转换为正弦位置编码

公式:

```
emb = timesteps * exp(-log(10000) * i / (dim/2 - 1))
emb = concat([sin(emb), cos(emb)])
```

3.5 适配器初始化函数

```
def init_adapter(adapter_module)
```

初始化策略:

组件	初始化方法
Conv2d权重	Kaiming正态分布 (mode='fan_out', nonlinearity='relu')
Conv2d偏置	常数 0.05
Linear权重	Xavier均匀分布
Linear偏置	常数 0.05 或 0.5（最后一层）
层级嵌入	正态分布 (mean=0, std=0.02)

3.6 训练主循环

```
while True:
    current_round += 1

    # 1. 发送权重和tau配置
    for n in range(n_nodes):
        send_msg(client_sock_all[n], ['MSG_WEIGHT_TAU_SERVER_TO_CLIENT', ...])

    # 2. 接收客户端更新
    for n in range(n_nodes):
        msg = recv_msg(client_sock_all[n],
'MSG_WEIGHT_TIME_SIZE_CLIENT_TO_SERVER')
        # 聚合参数和梯度

    # 3. 参数归一化
    for param in adapter_global.parameters():
        param.data /= data_size_total

    # 4. NaN检查和回滚
    if has_nan:
        adapter_global = w_global_prev

    # 5. 最小损失更新
    if loss_last_global < loss_min:
        loss_min = loss_last_global
        adapter_global_min_loss = deepcopy(w_global_prev)

    # 6. 计算下一轮tau
    tau_new, delt_f, gam_f = control_alg.compute_new_tau(...)

    # 7. 检查停止条件
    if stop_reason:
        break
```

3.7 梯度聚合结构

```
grad_global = {
    "img": {level: [] for level in adapt_layers},
    "img_hypernet": {
        "global_encoder": [],
        "level_predictors": {level: [] for level in adapt_layers},
        "level_embeddings": {level: [] for level in adapt_layers}
    }
}
```

3.8 能耗计算

```
# 计算传输能量
data_size_trans = sum(p.numel() * p.element_size() for p in adapter.parameters())
time_trans = data_size_trans * 8 / r[n]
E_trans = time_trans * p[n]
E_total += E_round
```

---

4. 客户端模块

文件: `client.py`

功能概述: 联邦学习客户端主程序，负责本地数据加载、模型训练、参数更新。

主要组件:

4.1 数据集类

```
class MedicalDataset(torch.utils.data.Dataset)
```

功能: 医学图像数据集封装

方法:

- `__getitem__(idx)`: 返回图像路径
- `__len__()`: 返回数据集大小

4.2 动量优化器

```
class HeavyBallMGD(torch.optim.Optimizer)
```

功能: Heavy Ball动量梯度下降优化器

参数:

参数	类型	默认值	说明
`params`	iterable	-	模型参数
`lr`	float	5e-4	学习率
`momentum`	float	0.9	动量系数

更新公式:

```
momentum_buffer = β * momentum_buffer + grad
param = param - lr * momentum_buffer
```

4.3 医学图像加载函数

```
def load_medical_image(image_path, modal_type="MRI")
```

功能：加载并预处理医学图像

支持格式：

- DICOM (.dcm)
- PNG, JPG, JPEG, BMP, TIF, TIFF

预处理步骤：

1. DICOM HU值转换
2. 窗宽窗位调整
3. 中值滤波去噪
4. 归一化到 [0, 255]

窗宽窗位配置：

模态	窗位	窗宽
CT	40	400
MRI	mean(img)	2 * std(img)

4.4 自适应数据预处理类

```
class AdaptiveResize(object)
```

功能：保持纵横比的图像缩放和填充

参数：

- `size`: 目标尺寸（正方形）
- `interpolation`: 插值方法
- `pad_value`: 填充值（默认0，黑色）

```
class AdaptiveWindowing(object)
```

功能：自适应窗宽窗位调整

逻辑：

- 较暗图像（mean < 100）：肺窗 (WC=-600, WW=1500)
- 较亮图像（mean >= 100）：纵隔窗 (WC=40, WW=400)
- 低对比度图像（std < 30）：对比度增强

4.5 潜变量生成函数

```
def image_to_latent_base(image_paths, vae, transforms, timesteps)
```

功能：生成指定时间步的带噪潜变量

流程：

1. 加载图像并预处理
2. VAE编码到潜空间
3. 添加噪声到指定时间步

4.6 能耗计算函数

```
def calculate_energy_consumption(tau_actual, accelerator, round_params, client_id)
```

功能：计算本地计算能耗

公式：

```
Ek = kappa * tau * batch_size * o * f²
```

4.7 本地训练流程

```
for tau in range(tau_config):
    # 1. 获取数据批次
    image_paths = next(loader_iter)

    # 2. 生成时间步
    timestep = random.choice(all_timesteps)

    # 3. 生成文本嵌入
    text_emb = text_encoder(tokenizer(text))

    # 4. 超网络生成kappa
    all_img_kappas = adapter["img_kappa_hypernet"](text_emb, timestep)

    # 5. 生成带噪潜变量
    noisy_latent, timesteps, noise = image_to_latent_base(...)
```

```
    # 6. 注册适配器钩子
    for level, layer in target_layers.items():
        hook = layer.register_forward_hook(create_hook(level, timesteps,
img_kappa))

    # 7. UNet前向传播
    noise_pred = unet(noisy_latent, timesteps, encoder_hidden_states=text_emb)

    # 8. 计算损失
    loss = F.mse_loss(noise_pred, noise)

    # 9. 反向传播
    loss.backward()

    # 10. 更新优化器
    img_adapter_optimizer.step()
    img_hypernet_optimizer.step()
```

4.8 梯度收集结构

```
current_grads = {
    "img": {level: [grad_tensors...] for level in adapt_layers},
    "img_hypernet": {
        "global_encoder": [grad_tensors...],
        "level_predictors": {level: [grad_tensors...] for level in adapt_layers},
        "level_embeddings": {level: [grad_tensors...] for level in adapt_layers}
    }
}
```

---

5. 控制算法模块

文件：`control_algorithm/adaptive_tau.py`

功能概述：自适应本地迭代次数控制算法，基于收敛界优化。

类：`ControlAlgAdaptiveTauServer`

```
class ControlAlgAdaptiveTauServer
```

初始化参数：

参数	类型	说明
`is_adapt_local`	bool	是否使用自适应本地迭代
`client_sock_all`	list	客户端Socket列表

参数	类型	说明
`n_nodes`	int	客户端数量
`control_param_phi`	float	控制参数φ
`moving_average_holding_param`	float	移动平均保持系数

内部状态:

属性	类型	说明
`beta_adapt_mvaverage`	float	beta移动平均值
`delta_adapt_mvaverage`	float	delta移动平均值
`omega_adapt_mvaverage`	float	omega移动平均值
`rho_adapt_mvaverage`	float	rho移动平均值
`tau_per_device`	dict	每个设备的tau值
`per_client_conv_params`	dict	每客户端收敛界参数

收敛界参数:

```
per_client_conv_params = {
    "Ld": [0.0] * n_nodes,      # Lipschitz常数
    "rho_cd": [0.0] * n_nodes,  # 损失ρ-Lipschitz常数
    "omega_cd": [0.0] * n_nodes, # ω_cd = 1/||θ_cd - θ*||²
    "s_d": [0.0] * n_nodes      # 数据量占比
}
```

5.1 偏差系数计算

```
def compute_tau_cd(self, t, alpha=0.1, beta=0.9, Ld=0.0)
```

功能: 计算附录D Lemma1的τ_[c]^d(t)

公式:

```
W = 1 + 3β + αLd
Z = 1 + β + αLd
X = √(Z² + 4β)
Y = 2β + αLd

τ_cd(t) = (W+X)/(2XY) * ((X+Z)/2)^t - (W-X)/(2XY) * ((Z-X)/2)^t - 1/Y
```

5.2 收敛界计算

```
def compute_epsilon0(self, tau_candidate, alpha=0.1, beta=0.9, client_idx=None)
```

功能：计算附录F Theorem3的ε0

公式：

```
ζ_cd = α(1 - αLd/2)

ε0 = [1 + √(1 + 4α * Σs_dλ_cdω_cdζ_cd * Σs_dδ_cdρ_cdτ_cd)] / (2Σs_dλ_cdω_cdζ_cd)

total_bound = ε0 + α * Σs_dδ_cdρ_cdτ_cd
```

5.3 Tau计算主函数

```
def compute_new_tau(self, data_size_local_all, data_size_total, tau, delt_f,
gam_f)
```

流程：

1. **接收客户端参数**：
   - beta_local (近似Ld)
   - rho_local (近似ρ_cd)
   - omega_local (近似ω_cd)
   - local_grad_global_weight
2. **计算梯度分歧δ_cd**：

   ```
   δ_cd = ||∇F_d(w) - ∇F(w)||₂
   ```

3. **移动平均更新**：

   ```
   param_mvavg = β * param_mvavg + (1-β) * param_new
   ```

4. **搜索最优tau**：
   - 对每个客户端独立搜索
   - 范围：[2, 8]
   - 目标：最小化收敛界

5. **返回结果**:
   - tau_new: 每个客户端的新tau值
   - delt_f, gam_f: 控制参数

类: `ControlAlgAdaptiveTauClient`

```
class ControlAlgAdaptiveTauClient
```

功能: 客户端控制算法辅助类

内部状态:

属性	类型	说明
`w_last_local_last_round`	ModuleDict	上一轮本地模型
`grad_last_local_last_round`	dict	上一轮梯度
`loss_last_local_last_round`	float	上一轮损失
`beta_adapt`	float	beta估计值
`rho_adapt`	float	rho估计值
`omega_adapt`	float	omega估计值

5.4 参数估计

```
def update_after_all_local(self, last_grad, last_loss, w, w_last_global,
loss_last_global, w_global_min_loss, omega)
```

估计公式:

参数	公式
`beta_adapt`	$\|\|\nabla F_d(w) - \nabla F(w)\|\|_2 / \|\|w_local - w_global\|\|_2$
`rho_adapt`	$\|\|loss_local - loss_global\|\| / \|\|w_local - w_global\|\|_2$
`omega_adapt`	$1 / \|\|w - w_min_loss\|\|^2$

---

6. 统计模块

文件: `statistic/collect_stat.py`

功能概述: 收集和记录训练统计数据

类：`CollectStatistics`

```
class CollectStatistics
```

初始化参数：

参数	类型	默认值	说明
`results_file_name`	str	"results.csv"	结果文件路径
`is_single_run`	bool	False	是否单次运行

6.1 CSV文件格式

单次运行 (SingleRun.csv)：

列名	说明
tValue	时间值
lossValue	损失值
betaAdapt	beta自适应值
deltaAdapt	delta自适应值
rhoAdapt	rho自适应值
tau	迭代次数

多次运行 (MultipleRuns.csv)：

列名	说明
tau_setup	tau设置
avg_tau	tau平均值
stddev_tau	tau标准差
avg_betaAdapt	beta平均值
stddev_betaAdapt	beta标准差
avg_deltaAdapt	delta平均值
stddev_deltaAdapt	delta标准差
avg_rhoAdapt	rho平均值
stddev_rhoAdapt	rho标准差
total_time_recomputed	总时间
E_total	总能耗

6.2 主要方法

```
def init_stat_new_global_round(self)
```

- 初始化新一轮统计变量

```
def collect_stat_end_local_round(self, tau, control_alg, total_time_recomputed,
loss_last_global)
```

- 收集每轮结束时的统计数据

```
def collect_stat_end_global_round(self, tau_setup, total_time,
total_time_recomputed, E_total)
```

- 收集全局轮次结束时的统计数据

---

7. 工具模块

文件：`util/utils.py`

功能概述：通用工具函数

7.1 消息发送函数

```
def send_msg(sock, msg)
```

功能：通过Socket发送消息

流程：

1. 序列化消息 (pickle)
2. 压缩数据 (zlib, level=3)
3. 发送总长度 (4字节)
4. 分块发送数据 (每块1MB)

7.2 消息接收函数

```
def recv_msg(sock, expect_msg_type=None, timeout=None)
```

功能：通过Socket接收消息

参数:

- `sock`: Socket对象
- `expect_msg_type`: 期望的消息类型
- `timeout`: 超时时间（秒）

流程:

1. 设置超时
2. 接收消息长度 (4字节)
3. 流式接收数据块
4. 解压缩数据
5. 反序列化消息
6. 验证消息类型

7.3 移动平均函数

```
def moving_average(param_mvavr, param_new, movingAverageHoldingParam)
```

公式:

```
param_mvavg = β * param_mvavg + (1-β) * param_new
```

7.4 数据分配函数

```
def get_indices_each_node_case(n_nodes, maxCase, label_list)
```

功能: 根据不同case分配数据到客户端

Case说明:

Case	分配策略
Case 0	均匀分配（轮询）
Case 1	按标签分配
Case 2	全局共享
Case 3	按标签范围分配

文件: `util/sampling.py`

功能概述: 小批量采样类

类： `MinibatchSampling`

```
class MinibatchSampling
```

初始化参数：

参数	类型	说明
`array`	list	数据数组
`batch_size`	int	批次大小
`sim`	int	随机种子偏移

方法：

```
def get_next_batch(self)
```

- 返回下一个批次的数据
- 数据用尽时自动重新打乱

文件： `util/time_generation.py`

功能概述：时间生成类

类： `TimeGeneration`

```
class TimeGeneration
```

初始化参数：

参数	类型	说明
`local_average`	float	本地时间均值
`local_stddev`	float	本地时间标准差
`local_min`	float	本地时间最小值
`global_average`	float	全局时间均值
`global_stddev`	float	全局时间标准差
`global_min`	float	全局时间最小值

方法：

```
def get_local(self, size)
```

- 生成本地时间（正态分布，截断到最小值）

```
def get_global(self, size)
```

- 生成全局时间（正态分布，截断到最小值）

---

8. 推理生成模块

文件：`apply/gen.py`

功能概述：使用训练好的适配器生成医学图像

命令行参数

参数	类型	默认值	说明
`--base_model_path`	str	"model"	基础模型路径
`--unet_subfolder`	str	"unets/4/unet"	UNet子目录
`--prompt`	str	训练文本	生成提示词
`--img_num`	int	1	生成图像数量
`--device`	str	"cuda:0"	设备
`--output_dir`	str	"image_result"	输出目录
`--num_inference_steps`	int	100	推理步数
`--seed`	int	42	随机种子
`--iteration_case`	str	"case1"	迭代case
`--momentum`	str	"with"	是否使用动量
`--config_case`	int	1	配置case
`--adapter_checkpoint_dir`	str	"output"	适配器检查点目录

生成流程

```
def main():
    # 1. 设置随机种子
    set_random_seed(args.seed)

    # 2. 加载UNet
```

```
    unet = UNetWithTimestepTrack.from_pretrained(...)

    # 3. 初始化适配器
    adapter = nn.ModuleDict()
    for level in adapt_layers:
        adapter[level] = nn.ModuleDict({
            "img_adapter": ImageAdapter(img_dim=img_dims_per_level[level])
        })
    adapter["img_kappa_hypernet"] = ImageKappaHyperNetwork(...)

    # 4. 加载训练好的权重
    checkpoint_path = os.path.join(args.adapter_checkpoint_dir, "adapter_best_tau-
1.pt")
    adapter.load_state_dict(torch.load(checkpoint_path))

    # 5. 初始化管道
    pipe = StableDiffusionPipeline.from_pretrained(...)

    # 6. 生成无适配器对比图
    for i in range(args.img_num):
        result = pipe(prompt=args.prompt, ...)
        image.save("without_adapter/without_adapter{i}.png")

    # 7. 预计算所有时间步的kappa
    all_timestep_kappas = {}
    for ts in scheduler.timesteps:
        img_kappas = adapter["img_kappa_hypernet"](text_embeddings, ts)
        all_timestep_kappas[ts.item()] = img_kappas

    # 8. 生成带适配器图像
    for i in range(args.img_num):
        # 注册钩子
        for level, layer in target_layers.items():
            hook = layer.register_forward_hook(create_adapter_hook_factory(level,
unet))

        # 生成图像
        result = pipe(prompt_embeds=adapted_text_emb, ...)
        image.save("with_adapter/{i}.png")
```

超参数详解

模型超参数

参数名	默认值	范围	说明
`resolution`	512	256-1024	输入图像分辨率
`train_batch_size`	8	1-32	训练批次大小
`learning_rate`	0.0003	1e-5 - 1e-3	学习率

参数名	默认值	范围	说明
gradient_accumulation_steps	1	1-8	梯度累积步数
mixed_precision	null	null/fp16/bf16	混合精度模式

联邦学习超参数

参数名	默认值	范围	说明
n_nodes	4	2-16	客户端数量
max_training_rounds	200	50-500	最大训练轮数
tau	2-8	2-8	本地迭代次数
momentum_value	0.9	0.5-0.99	动量系数
moving_average_holding_param	0.9	0.5-0.99	移动平均保持系数

适配器超参数

参数名	默认值	范围	说明
hidden_layers	2	1-4	隐藏层层数
hidden_size	[256, 128]	64-512	隐藏层大小
activation	"gelu"	relu/gelu/sigmoid/tanh	激活函数
kappa_scale	1	0.1-10	kappa缩放系数

时间步配置

参数名	默认值	范围	说明
timestep_range	[500, 601]	0-1000	时间步范围
num_inference_steps	100	20-1000	推理步数

停止条件

参数名	默认值	范围	说明
time_limit	null	-	时间上限（秒）
energy_limit	null	-	能量上限（焦耳）
convergence_rounds	20	5-50	收敛轮数
loss_improvement_percentage	1.0	0.1-5.0	损失改善阈值(%)

控制算法参数

参数名	默认值	范围	说明

参数名	默认值	范围	说明
`control_param_phi`	0.00005	1e-6 - 1e-3	控制参数φ
`alpha`	0.1	0.01-0.5	学习率α（收敛界计算）
`beta`	0.9	0.5-0.99	动量系数β（收敛界计算）

通信仿真参数

参数名	范围	单位	说明
`b` (带宽)	0.5-1	MHz	子信道带宽
`p_dBm` (发射功率)	5-20	dBm	发射功率
`f` (CPU频率)	0.5-1	GHz	CPU频率
`kappa` (能耗系数)	1e-28 - 5e-28	$J \cdot s^2/cycle^2$	计算能耗系数
`o` (CPU周期)	3e7 - 5e7	cycles	单样本CPU周期

数据流与通信协议

消息类型

消息类型	方向	内容
`MSG_INIT_SERVER_TO_CLIENT`	S→C	控制算法实例、use_min_loss、适配器参数、客户端ID
`MSG_DATA_PREP_FINISHED_CLIENT_TO_SERVER`	C→S	数据准备完成通知
`MSG_WEIGHT_TAU_SERVER_TO_CLIENT`	S→C	适配器参数、tau配置、是否最后一轮、最小损失模型、全局梯度
`MSG_WEIGHT_TIME_SIZE_CLIENT_TO_SERVER`	C→S	本地适配器、时间、tau、数据量、损失、梯度
`MSG_INFO_CLIENT_TO_SERVER`	C→S	本地计算能耗
`MSG_CONTROL_PARAM_COMPUTED_CLIENT_TO_SERVER`	C→S	控制参数计算状态
`MSG_BETA_RHO_GRAD_CLIENT_TO_SERVER`	C→S	beta、rho、梯度、omega

通信流程图

```
服务器                                    客户端
  |                                         |
  |——— MSG_INIT_SERVER_TO_CLIENT ————————>|
  |                                         | （初始化适配器）
  |                                         | （加载数据集）
  |<—— MSG_DATA_PREP_FINISHED ——————————|
```

```
    │                                          │
    │          ┌── 训练循环 ──┐                │
    │          │              │                │
    │──── MSG_WEIGHT_TAU ──────────────────────>│
    │                                          │ (本地训练)
    │<── MSG_WEIGHT_TIME_SIZE ─────────────────│
    │<── MSG_INFO (能耗) ──────────────────────│
    │<── MSG_CONTROL_PARAM ────────────────────│
    │<── MSG_BETA_RHO_GRAD ────────────────────│
    │          │              │                │
    │          └──────────────┘                │
    │                                          │
    │         (重复直到终止条件)                │
    │                                          │
```

使用指南

环境准备

```
# 安装依赖
pip install -r requirements.txt
```

数据准备

将医学图像数据放置在以下目录结构：

```
dataset_case4/
├── 1/          # 客户端1的数据
│   ├── image1.png
│   ├── image2.png
│   └── ...
├── 2/          # 客户端2的数据
│   └── ...
├── 3/          # 客户端3的数据
│   └── ...
└── 4/          # 客户端4的数据
    └── ...
```

启动训练

```
# 使用启动脚本
python start_training.py

# 或手动启动
# 终端1：启动服务器
```

```
python server.py

# 终端2-5：启动客户端
python client.py
```

生成图像

```
python apply/gen.py \
    --prompt "COVID-19 chest CT: ..." \
    --img_num 10 \
    --adapter_checkpoint_dir "output/dataset_case4_with_momentum"
```

依赖环境

核心依赖

包名	版本要求	用途
pytorch	>=1.13.0	深度学习框架
torchvision	>=0.14.0	图像处理
transformers	>=4.24.0	HuggingFace模型
diffusers	>=0.12.0	扩散模型
accelerate	>=0.14.0	分布式训练

图像处理

包名	版本要求	用途
Pillow	>=9.0.0	图像IO
opencv-python	>=4.5.0	图像处理
pydicom	-	DICOM文件处理

数据处理

包名	版本要求	用途
numpy	>=1.22.0	数值计算
scipy	>=1.8.0	科学计算
scikit-image	>=0.19.0	图像处理

监控与可视化

包名	版本要求	用途
matplotlib	>=3.5.0	可视化
wandb	>=0.13.0	实验追踪

参考文献

本项目的控制算法基于以下理论：

1. **收敛界优化**：附录D-F中的收敛界分析和tau优化策略
2. **Heavy Ball动量**：动量梯度下降的理论基础
3. **联邦学习**：分布式优化的参数聚合策略

许可证

本项目仅供学术研究使用。